

AthenaK Newtonian HD Local Adaptive Timestepping Implementation Note

LAT branch implementation summary

October 5, 2026

1 Scope

This note documents the local adaptive timestepping (LAT) implementation for Newtonian hydrodynamics in this AthenaK-derived code. It supports AMR and SMR, self-gravity through a frozen-potential window, an external black-hole source, and sink particles. The TDE example decks are the main use case. User-facing usage is described in the wiki page Local-Adaptive-Time-Stepping.

The driver refuses LAT for modules that are not aware of the active mask, namely viscosity, conduction, orbital advection, shearing boxes, turbulence driving, legacy radiation, Z4c, ion-neutral, particles, `<remap>`, and the integrators other than `rk1`, `rk2` and `imex2`. User source, time-step and fixup hooks are refused unless the problem generator flags them LAT-safe, and user boundary conditions are refused with `<hydro>`. This restriction is intentional, since every operator that changes state must know which MeshBlocks are active at the current local time. LAT for MHD and GRMHD exists only in the development version of the code.

The main implementation files are:

- `src/driver/driver.cpp` and `src/driver/driver.hpp`: LAT window scheduling, active-mask lifetime, synchronized gravity/AMR/output ordering, and restart/post-AMR LAT migration.
- `src/mesh/mesh.cpp`: local-CFL metadata, factor binning, same-level/AMR neighbor limiting, and cached bin counts.
- `src/mesh/meshblock_pack.cpp` and `src/mesh/meshblock_pack.hpp`: active MeshBlock masks, boundary-provider masks, flux-correction masks, mask caches, and per-block LAT time intervals.
- `src/mesh/load_balance.cpp`: LAT factor estimates for initial load balance, default LAT GID interleaving, per-timelevel partition objective, and cap-aware load balancing.
- `src/mesh/build_tree.cpp`, `src/mesh/mesh_refinement.cpp`, and `src/mesh/mesh_refinement.hpp`: startup/restart ordering, AMR candidate ordering, migration maps, and safe state transfer under non-Z-order GIDs.
- `src/pgen/pgen.cpp`: restart payload reads through the current-GID to restart-file-GID map.
- `src/hydro/hydro_tasks.cpp`, `src/hydro/hydro_update.cpp`, `src/hydro/hydro_newdt.cpp`, `src/hydro/hydro_fluxes.cpp`, and `src/hydro/hydro_dual_energy.cpp`: active-mask-aware hydro kernels, delayed refluxing, and dual-energy consistency.
- `src/bvals/`: time-aware cell-centered boundary exchange, prolongation/restriction, and AMR/same-level flux correction.

- `src/pgen/tde_external.cpp`: LAT-safe TDE source hooks and the black-hole sink/excision LAT factor cap.

2 User Controls

The main controls are in the `<time>` and `<mesh_refinement>` blocks:

Parameter	Meaning
<code>time/lat</code>	Enable HD LAT.
<code>time/lat_levels</code>	Number of binary local timestep levels (default 1, clamped to 1 to 20). The maximum synchronization factor is $2^{\text{lat_levels}}$.
<code>time/lat_same_level</code>	Permit same-AMR-level neighbors to use different LAT factors when boundary interpolation and same-level delayed refluxing are enabled.
<code>time/lat_same_level_max_ratio</code>	Maximum same-level factor ratio after a synchronized history is available.
<code>time/lat_neighbor_limiter</code>	Neighbor limiter scope: <code>all</code> , <code>hybrid</code> , or <code>face</code> .
<code>time/hydro_lat_min_bin_count</code>	Minimum useful bin population. <code>-1</code> means the default $4N_{\text{rank}}$; <code>0</code> disables sparse-bin pruning.
<code>time/hydro_lat_gid_reorder</code>	Enable LAT-optimized GID ordering. Default: <code>true</code> when LAT is enabled.
<code>time/hydro_lat_post_amr_rebalance</code>	Allow the driver to perform a migration-only LAT rebalance after AMR when needed. The default follows <code>hydro_lat_gid_reorder</code> .
<code>time/lat_diagnostics</code>	Print additional LAT bin and factor diagnostics.
<code>mesh_refinement/max_nmb_per_rank</code>	Hard GPU memory cap on MeshBlocks per rank, honored by LAT startup, restart, load balance, and AMR. There is no separate LAT cap.

When `lat=true`, the driver internally uses hydro subcycling with

$$\text{hydro_subcycle_factor} = 2^{\text{lat_levels}}.$$

This is an upper bound. Each LAT window chooses a possibly smaller synchronization factor from the available metadata, the remaining time to `tlim`, and the self-gravity cadence.

3 Basic Time Integration Algorithm

At a synchronized state all MeshBlocks are at the same physical time t^n . The code computes the ordinary global hydro timestep Δt_{fine} , which is the minimum local-CFL hydro timestep over all MeshBlocks. It then assigns each MeshBlock a power-of-two local factor

$$f_i \in \{1, 2, 4, \dots, F\},$$

where F is the active synchronization factor for the next LAT window. MeshBlock i advances with

$$\Delta t_i = f_i \Delta t_{\text{fine}}.$$

The driver then advances F fine ticks. At fine tick phase q , a block with factor f is due if

$$q \bmod f = 0.$$

Within each fine tick, due bins are visited from the largest factor down to $f = 1$. This coarsest-first sweep makes slow blocks provide boundary data before faster bins need it at the same fine-tick time.

At the end of every fine tick the mesh time and cycle advance by Δt_{fine} , not by the largest bin timestep. At the end of the LAT window all blocks are synchronized again. Only synchronized states are allowed to trigger AMR, output, after-cycle callbacks, local-CFL metadata rebuilds, or multigrid self-gravity source loading.

4 LAT Factor Metadata

`Hydro::NewTimeStep` stores local hydro timestep estimates in `dtnew_eachmb`. At synchronized states, `Mesh::UpdateHydroLATMetadata` gathers these estimates into `hydro_lat_dt_eachmb` and builds `hydro_lat_factor_eachmb`. The raw factor is the largest power-of-two value satisfying

$$f_i \Delta t_{\text{fine}} \leq \Delta t_{i,\text{local}},$$

clamped by the configured maximum factor.

The raw factors are then made safe by several reductions:

1. A problem-specific cap hook may lower factors when a problem generator installs one. The TDE problem generator currently does not install a LAT factor cap.
2. If `lat_same_level=false`, all blocks on an AMR level collapse to the minimum factor on that level.
3. Sparse bins are folded downward according to `hydro_lat_min_bin_count`. The default is $4N_{\text{rank}}$.
4. The neighbor limiter enforces a conservative staircase across AMR and same-level neighbors. The limiter is iterative and MPI-reduced: local changes are detected with a small allreduce first, and the full factor array is reduced only while changes still exist.

The final factor table is versioned with `hydro_lat_metadata_version`. `MeshBlockPack` active-mask caches are invalidated whenever metadata or topology changes. The mesh also stores `hydro_lat_bin_count`, so common per-tick queries such as “does bin f exist” are $O(1)$ instead of repeatedly scanning `nmb_total`.

5 Active Masks and Communication Masks

LAT is implemented by masking `MeshBlocks` inside one ordinary `MeshBlockPack`. The main arrays live in `MeshBlockPack`:

- `lat_active_mb` and `lat_active_indices`: blocks that execute state-changing hydro work in the current bin.
- `lat_boundary_send_mb` and `lat_boundary_send_indices`: inactive blocks that still need to provide ghost-zone data to active neighbors.

- `lat_send_nghbr`: per-neighbor boundary send mask.
- `lat_flux_accum_nghbr`, `lat_flux_send_nghbr`, and `lat_flux_recv_nghbr`: face-neighbor masks for delayed refluxing.
- `lat_time_start` and `lat_time_end`: the local time interval represented by each block’s current conserved state.

`SetActiveMeshBlocksByLATFactor` installs a single factor bin. `SetActiveMeshBlocksByLATDueFactors` installs the union of all bins due at a fine-tick phase. The union mask is used for the pre-bin boundary refresh at the tick start. Both routines compute and cache the boundary-need flags `lat_cached_needs_restrict`, `lat_cached_needs_prolongate`, `lat_cached_needs_physical_bc`, and `lat_cached_needs_neighbor_recv`. Therefore `RefreshHydroLATBoundaries` does not have to rescan all neighbors for every refresh.

It is important that “active” and “communication needed” are different. A rank may have no active `MeshBlocks` in a bin but still own boundary providers or flux receivers. In that case the relevant task lists still run on that rank so MPI send/receive ordering remains consistent.

6 Boundary Exchange and Time Interpolation

Cell-centered boundary exchange is time-aware under LAT. The driver sets `hydro_lat_exchange_time` before a refresh. Boundary packing and unpacking can use the block’s start state, end state, and requested target time to provide ghost data at the correct local time. Same-level mixed-factor boundaries use dense RK2 interpolation between the synchronized start state and the block’s current/end state.

The driver refresh sequence is:

1. Optional problem hydro state fixup at the target time.
2. Restriction if an active or provider block has a coarser neighbor that needs restricted data.
3. The usual `InitRecv`, `SendU`, `RecvU` boundary exchange and shear exchange.
4. Physical BCs, prolongation, and conserved-to-primitive conversion only when the cached mask says the operation is needed.

Non-fine bins may skip their final boundary exchange when the next fine tick will perform a union refresh. This avoids redundant communication without leaving synchronized states stale: the driver marks a boundary refresh as needed after each fine tick and performs the refresh at the next synchronized state before gravity, AMR, output, or timestep recomputation.

7 Delayed Flux Correction

Finite-volume conservation across mixed-timestep faces is handled by delayed refluxing. During a LAT bin, `Hydro::AccumulateLATCoarseFluxes` accumulates the slower/coarser side’s time-integrated face flux using the final RK flux weight and the local timestep. Faster/finer-side fluxes are sent through the existing AMR flux-correction buffers. When a receiver reaches the corresponding synchronization time, `Hydro::ApplyLATFluxCorrection` applies the flux difference.

This mechanism is used for both ordinary AMR coarse/fine faces and same-level mixed-factor faces when `lat_same_level=true`. Edge and corner neighbors participate in boundary exchange, but not flux correction, because finite-volume fluxes pass only through faces.

The reflux correction is split into independent x1, x2, and x3 directional kernels with fences between directions. This avoids races in corner/edge cells when multiple face corrections touch the same conserved cell. For dual-energy hydro, the auxiliary internal-energy/volume-fraction field receives a matching delayed correction. For self-gravity total-energy runs, the reflux path includes a gravitational energy consistency correction using the current potential.

8 Hydro, EOS, and Source-Term Coverage

All state-changing hydro kernels used by this LAT mode must respect `lat_active_mask_enabled`. The current implementation covers RK updates, flux computation, dual-energy synchronization, EOS conserved-to-primitive conversion, physical hydro boundary work, AMR prolongation/restriction, FOFC-related hydro paths, excision cleanup, and the TDE source term.

`Hydro::NewTimeStep` is not run inside masked LAT bins. Local-CFL estimates are meaningful only at synchronized states after boundary and reflux corrections are complete.

Generic user source terms are rejected unless the problem generator sets `user_srcs_lat_safe=true`. The TDE problem generator sets this only when `problem/use_translating_frame=false`. The translating frame uses global mesh-state-dependent acceleration and is therefore not LAT-safe. The active TDE gravity source loops over `lat_active_indices` when LAT is active and evaluates BH/frame data at the driver-provided stage/source time.

9 Self-Gravity

Self-gravity is synchronized with LAT. A LAT window reuses one synchronized potential; it does not run Poisson solves inside local bins, because doing so would load density from blocks at mixed local times. Before a LAT window, the driver refreshes the potential if it is invalid or due at the window start. It also shortens the proposed LAT window if the configured gravity cadence would require a solve before the proposed synchronization point.

At a synchronized LAT boundary the driver clears masks, refreshes hydro boundaries if delayed reflux changed any state, and then runs gravity if due. AMR and output see only synchronized hydro and gravity state.

10 LAT-Aware GID Ordering

The most important production load-balance change is the default LAT GID ordering. Standard AthenaK assigns GIDs by Z-order traversal, and each rank owns a contiguous GID interval. That contiguous interval assumption is used in many MPI and local-index calculations, so this branch does not assign arbitrary non-contiguous GIDs to ranks.

Instead, when `lat=true` and `hydro_lat_gid_reorder=true` (the default), the code changes the GID ordering itself. `BuildHydroLATGIDMap` groups candidate MeshBlocks by LAT factor, sorts each factor bin by Z-order for locality, and then emits a proportional interleaving of factor bins. `ApplyHydroLATGIDMapToArrays` applies that permutation to logical locations, costs, factor lists, optional restart maps, and the MeshBlockTree leaf GIDs.

This keeps the invariant

$$\text{local lid} = \text{gid} - \text{gids_eachrank}[\text{rank}]$$

valid everywhere. It also distributes fine-timestep factor-1 blocks across all ranks, instead of letting a compact BH/stream region occupy only one or two contiguous Z-order rank intervals. Existing

output readers continue to work because output carries each MeshBlock’s logical metadata and data; the internal GID order is not the physical plotting coordinate system.

11 LAT Load Balance Objective

After GID ordering, `Mesh::LoadBalance` still produces contiguous GID cuts. When factor metadata is available, the objective is per-timelevel rather than purely scalar. It builds prefix tables for:

- each factor bin count,
- scalar cost,
- work due at every tick phase in the LAT synchronization window.

The optimizer evaluates partitions by maximum per-tick work, sum of per-tick rank maxima, idle rank slots, factor-bin idle slots, maximum per-rank block count, block-count imbalance, and scalar cost. It tries several seeds: ordinary scalar cost, active work, equal blocks, factor-weighted costs, peak-tick weights, bin-balanced cuts, and critical-tick cuts. It then applies single-cut and adjacent-pair local search while enforcing `max_nmb_per_rank`.

If LAT metadata is unavailable, startup/restart load balance estimates factors conservatively from existing metadata, AMR containment, or AMR level. If even that is unavailable, it falls back to block-count balance so the memory cap remains a true allocation bound.

12 AMR and State Migration

AMR is allowed only at synchronized LAT states. The driver also suppresses a terminal AMR pass after the last accepted hydro update, since there is no following step to use the adapted mesh.

`MeshRefinement::RedistAndRefineMeshBlocks` has been made safe for LAT-reordered GIDs. For pure rebalance, it can request a new LAT GID map directly from the current logical-location list. For real AMR, it first lets the tree produce the usual candidate mesh and new-to-old map, then applies the LAT interleaving to both the candidate logical-location list and the `newtoold` migration map before `oldtonew`, load balance, MPI migration, and local in-place copies are built.

The AMR state-transfer routines were also adjusted for non-Z-order GIDs:

- `oldtonew` is built by logical containment rather than by assuming adjacent old/new GID arithmetic.
- Refine/derefine child lookup uses explicit child maps rather than relying on consecutive child GIDs.
- Local copy/rebalance routines handle arbitrary same-rank source/dest permutations with temporary storage for cycles.
- After topology changes, the MeshBlockTree leaf GIDs are reset from the final LAT-ordered logical-location array.

Hydro storage resize was changed to reduce transient GPU memory spikes during AMR. Persistent state arrays may retain capacity, while scratch and time-evolving arrays use `Kokkos::realloc` to avoid holding both old and new scratch allocations longer than needed.

13 Restart Semantics

LAT restarts require a single global restart file. Per-rank restart files are rejected when LAT is enabled because arbitrary GID reordering cannot safely read rank-local payload files without a global data map.

Restart handling has two distinct maps:

1. The restart file stores payloads by the GID order used when the file was written.
2. The live mesh may immediately reorder GIDs for LAT load balance.

Therefore `BuildTreeFromRestart` constructs `restart_gid_eachmb`, a current-GID to restart-file-GID map. If the startup LAT GID interleaving composes another permutation, that restart map is composed as well. `ProblemGenerator::ProblemGenerator` then reads every cell-centered and face-centered restart payload from

$$\text{headeroffset} + \text{restart_file_gid} \times \text{data_size} + \text{field_offset}.$$

The mapped path uses independent `MPI_File_read_at` calls, so ranks do not have to issue collective reads in identical GID order. After all payloads are read, the restart map is deleted.

On restart, the driver computes a fresh timestep and LAT metadata. If the mesh is already LAT-reordered, it keeps that ordering. If not, it may perform a migration-only `RedistAndRefineMeshBlocks(pin, 0, 0)` pass to move the live state into the LAT-optimized ordering before normal evolution.

14 TDE-Specific Sink and Source Safety

The TDE problem generator no longer installs a BH-specific LAT factor cap. Sink-adjacent blocks are handled by the normal LAT factor pipeline and neighbor limiter.

The BH gravity and floor-shell cleanup source loops use the LAT active-index list when LAT is active. The source time comes from the driver's stage/source time hook. Translating-frame runs are marked LAT-unsafe, because the frame acceleration depends on global gas state and would otherwise be evaluated inconsistently across local times.

15 Performance Notes

The current branch includes several performance-oriented changes that are important to production scaling:

- Factor-bin counts are cached in `hydro_lat_bin_count`, avoiding repeated full-array scans during each LAT tick.
- Active-mask and due-factor masks are cached by metadata version, maximum factor, tick phase, and neighbor-table size.
- Boundary refresh gating uses cached need flags computed during mask construction rather than rescanning neighbors on every refresh.
- Neighbor limiter convergence first reduces a small changed flag before reducing the full factor array.

- The LAT load balancer prints per-bin and per-tick efficiency diagnostics when active, making fine-tick rank imbalance directly visible.

The dominant cost that remains is communication and task-list launch overhead for frequent boundary refreshes at large synchronization factors. The design intentionally favors correctness and synchronized AMR/gravity behavior over fully asynchronous per-rank progress.

16 Example Configuration

The example deck `inputs/TDE_examples/tde_05_fallback_lat.athinput` uses:

- `lat` (set to `true` to enable LAT),
- `lat_levels=7`, giving a configured maximum factor $2^7 = 128$,
- `lat_same_level=true`,
- `lat_same_level_max_ratio=2`,
- `lat_neighbor_limiter=all`,
- `default_hydro_lat_gid_reorder=true`,
- `gravity/solve_dt=0.03`,
- `mesh_refinement/max_nmb_per_rank=520`.

The exact factor distribution is runtime-dependent because it follows the local-CFL table and AMR topology.

17 Limitations and Invariants

This implementation should not be treated as a full H-AMR replacement. Its current limitations are:

- Newtonian HD only in this release.
- RK1/RK2 are the validated integrators for the delayed-reflux path.
- AMR, gravity, output, callbacks, and timestep metadata are synchronized at LAT window boundaries, not run asynchronously in local bins.
- Same-level temporal interpolation is dense RK2 between start and end conserved states; very large same-level ratios remain a physics validation risk.
- Per-rank restart files are not supported with LAT.

Future LAT changes should preserve these invariants:

1. Never run AMR, output, self-gravity source loading, or after-cycle callbacks on a partially synchronized LAT mask.
2. Recompute local-CFL LAT metadata only at synchronized states.

3. Keep active evolution masks separate from boundary-provider and flux-receiver masks.
4. Keep MeshBlock GID ownership contiguous per rank; use GID ordering permutations rather than arbitrary non-contiguous rank assignments.
5. Compose restart payload maps whenever startup GIDs are permuted.
6. Keep the AMR `newtoold/oldtonew` maps consistent with any LAT candidate ordering before MPI migration or local copies.
7. Preserve the hard `max_nmb_per_rank` cap and `NUM_BITS_LID` tag limit.
8. Do not mark a user source LAT-safe until every state-changing loop is active-mask-aware and all time-dependent data use the driver-provided stage/source time.
9. If same-level LAT behavior is extended, update boundary interpolation and delayed refluxing together.